

Análisis de resultados

Nombre: Alejandra del Rosario Ramos Mamani

Código: 2024-119036

1. **¿Cuáles fueron las principales diferencias observadas en el rendimiento del código con hilos en comparación con un programa monohilo y cómo influyó el multihilo en el tiempo de ejecución?**

En la comparación de rendimiento, el programa monohilo ejecuta las instrucciones de manera estrictamente secuencial, mientras que la versión multihilo permite paralelizar tareas dividiendo el trabajo entre los distintos núcleos del procesador, lo que reduce drásticamente el tiempo de ejecución en tareas de cálculo intensivo o con bloqueos de Entrada/Salida, sin embargo, para operaciones extremadamente cortas, el programa monohilo puede resultar ligeramente más rápido debido a que la versión multihilo incurre en una sobrecarga (*overhead*) de tiempo y recursos al tener que crear, coordinar y destruir cada hilo.

2. **¿Qué funciones específicas de la biblioteca pthread utilizaste y cuál es la finalidad de cada una en el programa?**

En la práctica se utilizaron la función `pthread_create()` para solicitar al sistema operativo la creación e inicio de ejecución de un nuevo hilo, `pthread_join()` para pausar y sincronizar el hilo principal esperando a que los hilos secundarios terminen, `pthread_mutex_init()` y `pthread_mutex_destroy()` para inicializar y liberar la estructura del mutex, `pthread_mutex_lock()` y `pthread_mutex_unlock()` para bloquear y desbloquear el acceso exclusivo a la sección crítica garantizando la exclusión mutua, y finalmente `pthread_exit()` para finalizar la ejecución de los hilos de manera limpia.

3. **¿Cómo afectó el incremento en el número de hilos (1, 5 y 10) al rendimiento del programa y si este mejoró de forma lineal?**

El incremento en la cantidad de hilos de 1 a 5 y luego a 10 no produjo una mejora lineal en el rendimiento, sino que incrementó el tiempo total de ejecución debido a la naturaleza ligera y corta de la tarea, esto sucede porque el paralelismo se ve anulado por la sobrecarga que genera el sistema operativo al realizar constantes cambios de contexto (*context switching*) entre hilos, sumado a la alta contención que ocurre cuando múltiples hilos se bloquean esperando su turno para adquirir el único mutex que protege la salida estándar.

4. **¿Observaste alguna condición de carrera o problema de concurrencia durante la ejecución del programa sin el uso de mutex y cómo influyó la sincronización?**

Sin el uso de mutex, se observaron condiciones de carrera traducidas en una salida por consola impredecible y desordenada, donde el texto impreso por diferentes hilos se intercalaba al competir simultáneamente por el buffer compartido de la pantalla, la implementación de los mecanismos de sincronización solucionó este problema al forzar la exclusión mutua, garantizando que un solo hilo tome el control del recurso compartido a la vez y eliminando cualquier corrupción o cruce de datos durante la ejecución.